

Selenium to Playwright conversion

Pages files

The screenshot shows the GitHub web interface for the repository 'Harrish-sdet / CRM'. The breadcrumb path is 'CRM / CRM_25-main / CRM_25-main / src / main / java / com / salesforce / pages'. The left sidebar shows the file tree with 'pages' selected under 'salesforce'. The main content area shows a commit history table for the 'pages' directory.

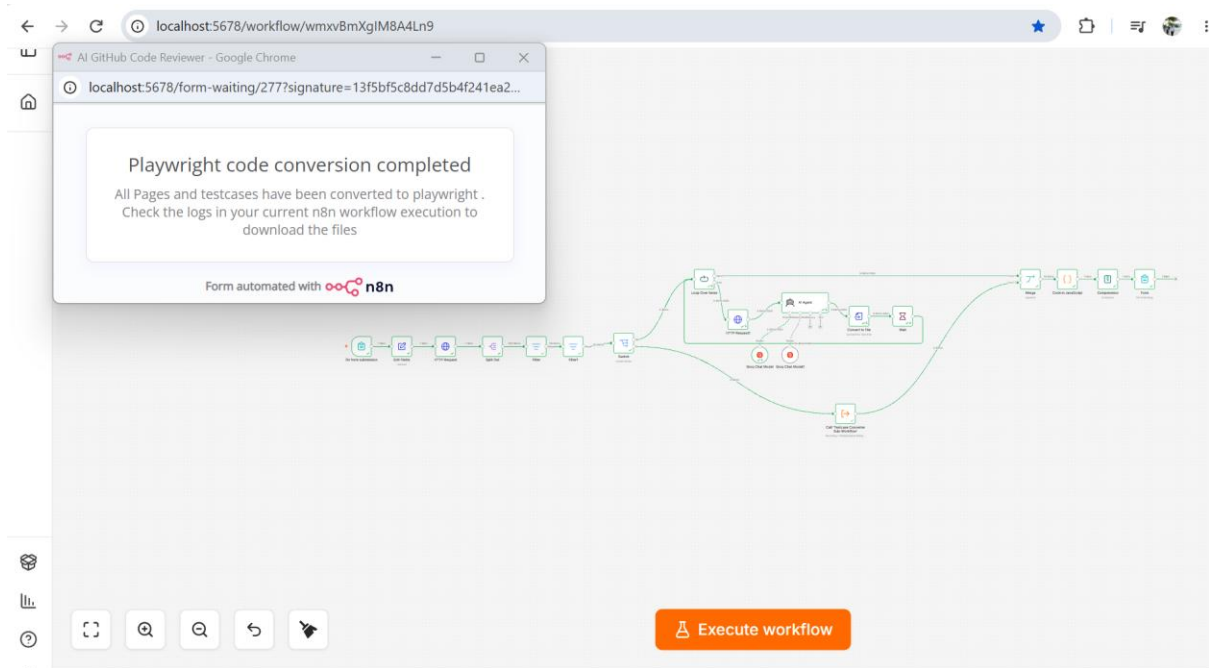
Name	Last commit message	Last commit date
..		
CasePage.java	first commit	5 days ago
HomePage.java	first commit	5 days ago
IndividualsPage.java	first commit	5 days ago
LoginPage.java	first commit	5 days ago
WorkTypeGroupPage.java	first commit	5 days ago

Testcases files

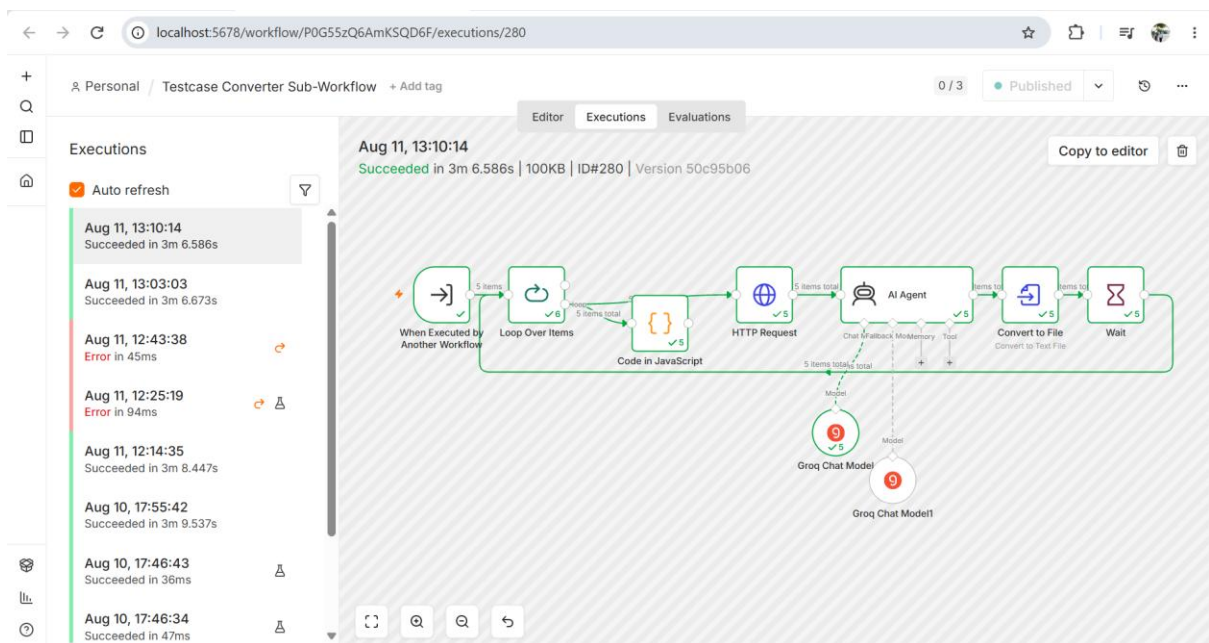
The screenshot shows the GitHub web interface for the repository 'Harrish-sdet / CRM'. The breadcrumb path is 'CRM / CRM_25-main / CRM_25-main / src / main / java / com / salesforce / testcases'. The left sidebar shows the file tree with 'testcases' selected under 'salesforce'. The main content area shows a commit history table for the 'testcases' directory.

Name	Last commit message	Last commit date
..		
TC001_CreateNewCase.java	first commit	5 days ago
TC002_CreateIndividuals.java	first commit	5 days ago
TC003_EditIndividuals.java	first commit	5 days ago
TC004_DeleteIndividuals.java	first commit	5 days ago
TC005_CreateWorkType.java	first commit	5 days ago

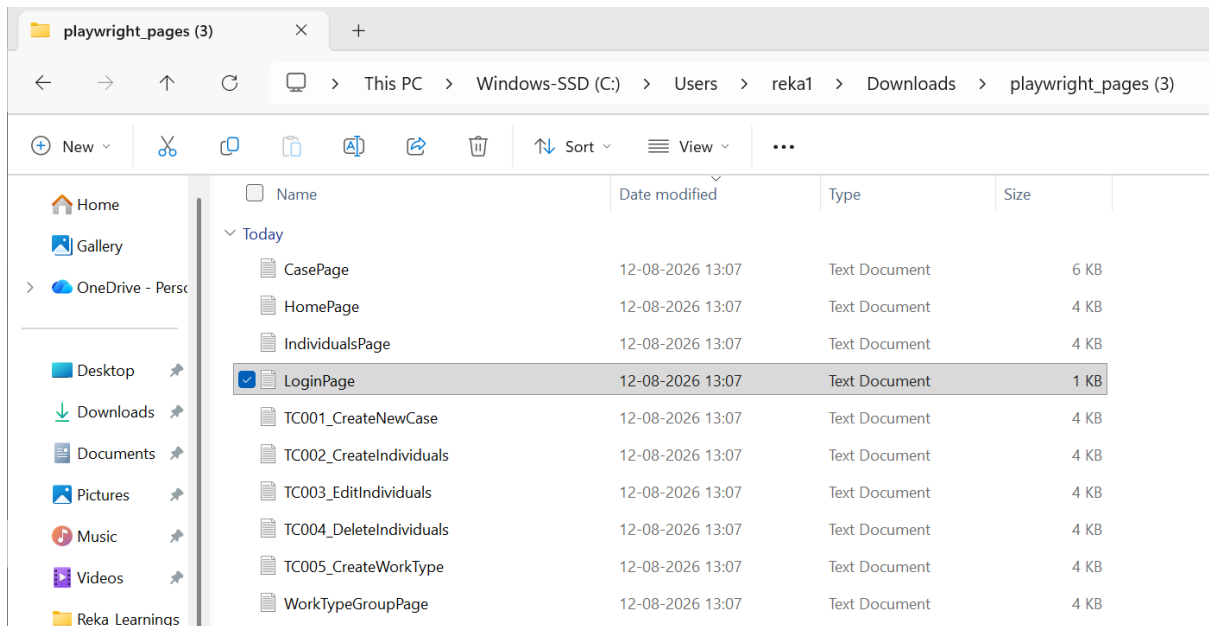
Main workflow – 5 pages + 5 test cases = 10 items processed



Sub workflow (Test case converter)



Output files (Playwright converted)



Sample file content

Page:

```
File Edit View H1 [Icons] B I S [Icons] [Icons] [Icons] Login X

``typescript
import { BrowserContext, Page } from "@playwright/test";
import { PlaywrightWrapper } from "../helpers/playwright";

export class LoginPage extends PlaywrightWrapper {
  private readonly selectors = {
    usernameField: "#username",
    passwordField: "#password",
    loginButton: "#Login",
  };

  constructor(page: Page, context: BrowserContext) {
    super(page, context);
  }

  public async enterUsername(username: string) {
    await this.type(this.selectors.usernameField, "Username Field", username);
  }

  public async enterPassword(password: string) {
    await this.type(this.selectors.passwordField, "Password Field", password);
  }

  public async clickLogin() {
    await this.click(this.selectors.loginButton, "Login Button", "Button");
  }
}
```

Test case

```
data.txt TC002_Create LoginPage.txt TC001_Create TC004_Delete output (1).txt output (2).txt output.txt TC002_Create TC003_EditIn TC004_Delete TC004 x
File Edit View H1 ≡ B I S ⇄ ⌂
```typescript
import { BrowserContext, Page } from "@playwright/test";
import { PlaywrightWrapper } from "../helpers/playwright";

export class LoginPage extends PlaywrightWrapper {
 private readonly selectors = {
 usernameField: "#username",
 passwordField: "#password",
 loginButton: "#login",
 };

 constructor(page: Page, context: BrowserContext) {
 super(page, context);
 }

 public async enterUsername() {
 await this.type(this.selectors.usernameField, "Username Field", "");
 }

 public async enterPassword() {
 await this.type(this.selectors.passwordField, "Password Field", "");
 }

 public async clickLogin() {
 await this.click(this.selectors.loginButton, "Login", "Button");
 }
}

export class AppLauncherPage extends PlaywrightWrapper {
 private readonly selectors = {
 appLauncherIcon: ".slds-icon-waffle",
 viewAllBtn: "//button[text()='View All']",
 searchField: "one-app-launcher-modal input.slds-input",
 workTypeGroupLink: "//a[@title='Work Type Groups']",
 };

 constructor(page: Page, context: BrowserContext) {
 super(page, context);
 }
}
```

Ln 1, Col 1 | 3,185 characters | Plain text | 100% | Unix (LF)

## High-Level Overview: Selenium to Playwright Workflow

This n8n workflow automates the conversion of legacy Selenium test cases into modern Playwright code using generative AI. It fetches code files from a repository, processes them either iteratively through a loop or via a modular sub-workflow, translates the code using an AI model, and packages the results into downloadable files.

### Step-by-Step Execution Pipeline

#### 1. Trigger & Data Ingestion

- **On form submission:** Initiates the workflow manually via a form input.
- **HTTP Request:** Connects to the source repository to fetch the list of files.

#### 2. Filtering & Routing

- **Split Out & Filters (Filter, Filter1):** Flattens the file list array and strips out unwanted files, keeping only the targeted test scripts.
- **Switch:** Directs traffic based on predefined rules—routing items either through an internal processing loop or offloading them to a sub-workflow.

#### 3. Core Processing (Loop Path)

- **Loop Over Items:** Processes files sequentially.
- **HTTP Request1:** Retrieves the raw source code of the current file.

- **AI Agent (powered by Groq Chat Models):** Analyzes the legacy Selenium code and rewrites it into Playwright syntax.
- **Convert to File:** Packages the AI-generated code into formatted text files.
- **Wait:** Manages execution pacing between iterations.

#### 4. **Alternative Processing (Sub-Workflow Path)**

- **Call 'Testcase Converter Sub-Workflow':** Encapsulates the conversion logic in a modular sub-workflow (P0G55zQ6AmKSQD6F) for streamlined execution.

#### 5. **Consolidation & Completion**

- **Merge:** Combines processed outputs from the branches back into a single stream.
- **Code in JavaScript:** Applies final formatting and structure cleanups.
- **Form Ending:** Concludes the workflow run and delivers the final output.